

Memgraph-Anleitung

Verbindung, Graph-Struktur, Cypher-Queries und Demo-Schritte 0–4

Masterprojekt · Hochschule RheinMain · September 2026

Diese Anleitung erklärt, wie Memgraph in CRYME genutzt wird: Verbindung, Graph-Struktur, Cypher-Queries und der Zusammenhang mit den Demo-Migrationsschritten 0–4. Technischer Code-Überblick: **Technische Dokumentation** · Graph-Versionierung: docs/GRAPH_VERSIONING.md.

1. Was Memgraph in CRYME macht

Memgraph ist die **Persistenzschicht** für:

1. Den **digitalen Zwilling** (Knoten + Kanten aus `webserver_pqc_twin.yaml`)
2. Den **Laufzeitstatus** (`status`, `active_algorithm` pro Knoten)
3. Die **Migrationshistorie** (`MigrationStep-Events`, `HEAD-Zeiger`)

CRYME nutzt das **Bolt-Protokoll** (kompatibel mit `neo4j-driver`) und **Cypher-Queries**.

Die Oracle in `web_app/oracle.js` führt Graph-Algorithmen **nicht** als Memgraph-Prozeduren aus. Sie lädt den Graphen in den Speicher, berechnet SCC/Validierung in JavaScript und schreibt das Ergebnis per Cypher zurück.

2. Verbindung herstellen

2.1. Lokal (Mac / Linux)

```
docker compose -f deploy/docker-compose.yml up -d
bash deploy/check_memgraph.sh
```

Dienst	Adresse	Zugangsdaten
Bolt (CLI / Driver)	<code>bolt://localhost:7687</code>	kein User, kein Passwort
Memgraph Lab (GUI)	<code>http://localhost:3000</code>	Connect: <code>bolt://localhost:7687</code>

2.2. Über SSH-Tunnel (Universitätsserver ilmare)

```
ssh -L 3000:127.0.0.1:3000 -L 7687:127.0.0.1:7687 admin@ilmare
```

Im Browser: `http://localhost:3000` → Verbindung zu `bolt://localhost:7687`

2.3. Über die CRYME-CLI (ohne GUI)

```

cryme show node          # alle Crypto-Knoten
cryme show tree          # Migrationshistorie
cryme show state step=2  # Zustand bei Schritt 2
cryme show graph step=2  # Abhängigkeitsgraph bei Schritt 2

```

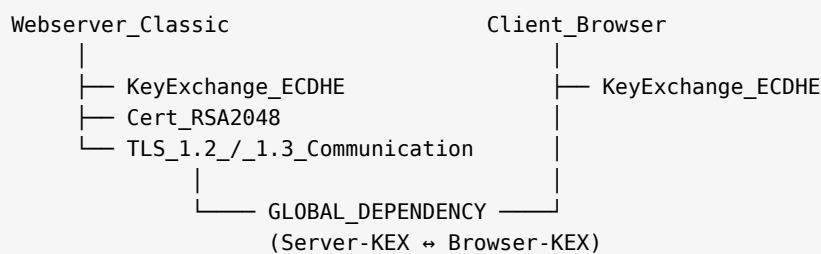
2.4. Node.js-Verbindungstest

```
cd web_app && node check_db.js
```

Gibt alle MigrationStep-Knoten und TRANSITION_T0-Kanten aus.

3. Graph-Übersicht (CRYME-Demo)

Nach `cryme init` enthält der Graph:



3.1. Knotenlabels

Label	Anzahl (Demo)	Beispiel-ID
Component	2	Webserver_Classic
CryptoAsset	3	Webserver_Classic.KeyExchange_ECDHE
SecurityControl	1	Webserver_Classic.TLS_1.2/_1.3_Communication
PQCVariant	mehrere	Zielalgorithmen pro Asset
MigrationStep	wächst mit jeder Migration	step=0, 1, 2, ...
SystemMeta	1 (nach erster migrate)	head_step-Zeiger

3.2. Kantentypen

Relation	Bedeutung	Sichtbar in
HAS_ASSET	Component → CryptoAsset	Struktur
HAS_CONTROL	Component → SecurityControl	Struktur
HAS_VARIANT	CryptoAsset → PQCVariant	Struktur
GLOBAL_DEPENDENCY	Server-KEX ↔ Browser-KEX	<code>cryme show graph</code>
IMPLICIT_DEPENDENCY	SecurityControl → CryptoAsset	<code>cryme show graph</code>
EXPLICIT_DEPENDENCY	Deklarierte funktionale Abhängigkeit	<code>cryme show graph</code>
TEMPORAL_CONSTRAINT	Phasenordnung der Komponenten	Struktur

Relation	Bedeutung	Sichtbar in
TRANSITION_TO	Schritt N → Schritt N+1	cryme show tree

4. Memgraph Lab — Erste Schritte

4.1. Gesamten Graph anzeigen

1. `http://localhost:3000` öffnen
2. Quick connect: `bolt://localhost:7687`
3. Query ausführen:

```
MATCH (n)
RETURN n
LIMIT 80;
```

4.2. Alle CryptoAssets

```
MATCH (a:CryptoAsset)
RETURN a.id AS asset,
       a.status AS status,
       a.algorithm AS baseline,
       a.active_algorithm AS active
ORDER BY a.id;
```

4.3. Abhängigkeitsgraph

```
MATCH (a)-[:GLOBAL_DEPENDENCY|EXPLICIT_DEPENDENCY|IMPLICIT_DEPENDENCY]->(b)
RETURN a.id AS from, type(r) AS rel, b.id AS to;
```

Die GLOBAL_DEPENDENCY zwischen Server- und Browser-KEX ist der Grund für den **Fehler in Schritt 1**.

4.4. PQC-Zielvarianten anzeigen

```
MATCH (a:CryptoAsset)-[:HAS_VARIANT]->(v:PQCVariant)
RETURN a.id AS asset, v.algorithm AS target_algo, v.security_level AS level
ORDER BY a.id, v.algorithm;
```

5. Migrationshistorie abfragen

5.1. Alle Schritte

```
MATCH (s:MigrationStep)
RETURN s.step AS step,
       s.status AS status,
       s.action AS action,
       s.head AS head
ORDER BY s.step;
```

Entspricht: `cryme show tree`

5.2. Migrationsbaum

```
MATCH (a:MigrationStep)-[:TRANSITION_TO]->(b:MigrationStep)
RETURN a.step AS parent, b.step AS child
ORDER BY parent, child;
```

5.3. HEAD-Zeiger

```
MATCH (m:SystemMeta {id: 'cryme'})
RETURN m.head_step AS head_step;
```

```
MATCH (s:MigrationStep {head: true})
RETURN s.step AS head, s.status AS status, s.action AS action;
```

6. Demo-Schritte 0–4 in Memgraph

Zuerst den CLI-Befehl ausführen, danach die Cypher-Query daneben. Das ist die vorgesehene Paarung: **CLI verändert den Graphen, Lab inspiziert ihn.**

6.1. Schritt 0 — Baseline (cryme init)

CLI:

```
cryme init
cryme show state step=0
```

Cypher — alle Knoten classic?

```
MATCH (a:CryptoAsset)
RETURN a.id, a.status, a.algorithm, a.active_algorithm;
```

Knoten	status	algorithm	active_algorithm
Webserver_Classic.KeyExchange_ECDHE	classic	ECDHE	null
Webserver_Classic.Cert_RSA2048	classic	RSA-2048	null
Client_Browser.KeyExchange_ECDHE	classic	ECDHE	null

```
MATCH (s:MigrationStep {step: 0})
RETURN s.status, s.action, s.head;
```

Erwartung: Nach der ersten cryme migrate (MERGE auf Schritt 0): status = 'init'. Direkt nach cryme init existiert dieser Knoten oft noch nicht — geladen wird nur die Twin-Topologie.

Live-API (durch init zurückgesetzt, nicht durch migrate):

```
curl -sk https://127.0.0.1:8443/api/status | python3 -m json.tool
```

Erwartung: "migration_step": 0, "profile": "classic-rsa-ecdhe"

6.2. Schritt 1 — Isolierte Server-KEX-Migration (FAILED)

CLI:

```
cryme migrate id=Webserver_Classic.KeyExchange_ECDHE X25519_MLKEM768
cryme show tree
```

Cypher — fehlgeschlagener Schritt:

```
MATCH (s:MigrationStep {step: 1})
RETURN s.status, s.action, s.cluster, s.discovered_edge;
```

Feld	Wert
status	failed
action	migrate_fail
cluster	enthält Webserver_Classic.KeyExchange_ECDHE
discovered_edge	Server↔Browser-Verbindung

Cypher — Knoten unverändert (noch classic):

```
MATCH (a:CryptoAsset {id: 'Webserver_Classic.KeyExchange_ECDHE'})
RETURN a.status, a.active_algorithm;
```

Erwartung: status = 'classic', active_algorithm = null

Cypher — neu entdeckte Kante (nach Oracle-Lernen):

```
MATCH (a)-[r:IMPLICIT_DEPENDENCY {discovered: true}]->(b)
RETURN a.id AS from, b.id AS to;
```

HEAD bleibt bei Schritt 0:

```
MATCH (m:SystemMeta {id: 'cryme'}) RETURN m.head_step;
```

Dieses Beispiel zeigt, warum Memgraph auch **fehlgeschlagene** Ereignisse speichert: die nächste Migration nutzt die entdeckte implizite Kante, sodass beide KEX-Knoten eine SCC bilden.

6.3. Schritt 2 — Co-Migration Server + Browser KEX (SUCCESS)

CLI:

```
cryme migrate id=Webserver_Classic.KeyExchange_ECDHE X25519_MLKEM768
cryme deploy step=2
cryme verify step=2
```

Cypher — beide KEX migriert:

```
MATCH (a:CryptoAsset)
WHERE a.id CONTAINS 'KeyExchange'
RETURN a.id, a.status, a.active_algorithm, a.migrated_at_step;
```

Knoten	status	active_algorithm	migrated_at_step
Webserver_Classic.KeyExchange_ECDHE	migrated	X25519_MLKEM768	2
Client_Browser.KeyExchange_ECDHE	migrated	X25519_MLKEM768	2

Cypher — Schritt-Details:

```
MATCH (s:MigrationStep {step: 2})
RETURN s.status, s.cluster, s.variants, s.head;
```

Erwartung: status = 'success', head = true, cluster enthält beide KEX-Knoten

Live-API nach deploy:

```
curl -sk https://127.0.0.1:8443/api/status | python3 -m json.tool
```

Erwartung: "migration_step": 2, KEX-Algorithmen = X25519_MLKEM768

6.4. Schritt 3 — Zertifikat ML-DSA-44 (SUCCESS)

CLI:

```
cryme migrate id=Webserver_Classic.Cert_RSA2048 ML-DSA-44
cryme deploy step=3
```

```
MATCH (a:CryptoAsset {id: 'Webserver_Classic.Cert_RSA2048'})
RETURN a.status, a.algorithm, a.active_algorithm, a.migrated_at_step;
```

Erwartung: status = 'migrated', active_algorithm = 'ML-DSA-44', migrated_at_step = 3

Cypher — kumulativer Zustand (alle bisher migrierten Knoten):

```
MATCH (n)
WHERE (n:CryptoAsset OR n:SecurityControl) AND n.status = 'migrated'
RETURN n.id, n.active_algorithm, n.migrated_at_step
ORDER BY n.migrated_at_step;
```

Erwartung: 3 Knoten (2× KEX + Cert)

TLS-Verifikation (Zertifikat gewechselt):

```
echo | openssl s_client -connect 127.0.0.1:8443 2>/dev/null | openssl x509 -noout -subject
```

Erwartung: CN=CRYME Live ML-DSA Demo (ECDSA-Stand-in für ML-DSA)

6.5. Schritt 4 — TLS 1.3 only (SUCCESS, alles migriert)

CLI:

```
cryme migrate id=Webserver_Classic.TLS_1.2/_1.3_Communication TLS1.3
cryme deploy step=4
cryme show state step=4
cryme verify step=4
```

Cypher — SecurityControl:

```
MATCH (c:SecurityControl)
RETURN c.id, c.status, c.active_algorithm, c.migrated_at_step;
```

Erwartung: active_algorithm = 'TLS1.3', status = 'migrated'

Cypher — alle Knoten migriert?

```
MATCH (n)
WHERE (n:CryptoAsset OR n:SecurityControl)
RETURN n.id, n.status, n.active_algorithm
ORDER BY n.id;
```

Erwartung: alle 4 Knoten status = 'migrated'

Cypher — HEAD bei Schritt 4:

```
MATCH (s:MigrationStep {head: true})
RETURN s.step, s.status, s.action;
```

Live-API:

```
curl -sk https://127.0.0.1:8443/api/status | python3 -m json.tool
```

Erwartung: "migration_step": 4, "profile": "tls13-only"

Client-Simulation (curl-client Container):

```
docker exec cryme-curl-client sh -c './client/expect.env && curl -skv $CRYME_CURL_TLSFLAGS https://nginx-classic/health'
```

Erwartung: CRYME_CURL_TLSFLAGS="--tlsv1.3 --tls-max 1.3"

7. Zwei Sichten: Memgraph vs. Live-API

Sicht	Quelle	Wann aktualisiert
Planung	Memgraph (Replay)	sofort nach cryme migrate
Live	nginx /api/status	erst nach cryme deploy step=N

7.1. Beispiel: Schritt 2 migriert, aber nicht deployed

```
cryme show state step=2
# → KEX-Knoten: migrated

curl -sk https://127.0.0.1:8443/api/status
# → migration_step: 0 (noch nicht deployed!)
```

Nach `cryme deploy step=2` müssen beide übereinstimmen.

Prüf-Query — letzter erfolgreicher Schritt vs. Live:

```
MATCH (m:SystemMeta {id: 'cryme'})
RETURN m.head_step AS memgraph_head;
```

```
curl -sk https://127.0.0.1:8443/api/status | python3 -c "import sys,json;
print(json.load(sys.stdin)['migration_step'])"
```

Beide Werte sollten nach Deploy gleich sein.

8. Nützliche Queries für Gutachter

8.1. Anzahl Knoten pro Label

```
MATCH (n)
RETURN labels(n)[0] AS label, count(n) AS count
ORDER BY count DESC;
```

8.2. Varianten eines Assets

```
MATCH (a:CryptoAsset {id: 'Webserver_Classic.KeyExchange_ECDHE'})-[:HAS_VARIANT]->(v)
RETURN v.algorithm, v.security_level, v.performance;
```

8.3. Fehlgeschlagene Schritte

```
MATCH (s:MigrationStep {status: 'failed'})
RETURN s.step, s.action, s.cluster, s.discovered_edge;
```

8.4. Erfolgreiche Schritte mit Algorithmen

```
MATCH (s:MigrationStep {status: 'success'})
RETURN s.step, s.cluster, s.variants
ORDER BY s.step;
```

8.5. Graph visualisieren (nur Abhängigkeiten)

```
MATCH p=(a)-[r:GLOBAL_DEPENDENCY|IMPLICIT_DEPENDENCY|EXPLICIT_DEPENDENCY]->(b)
RETURN p;
```

9. Graph zurücksetzen

```
cryme init
```

Intern:

```
MATCH (n) DETACH DELETE n;
```

Dann lädt `twin_loader.js` den Zwilling neu aus YAML. **Alle Migrationshistorie geht verloren** — für Demos gewollt.

10. Häufige Probleme

Problem	Ursache	Lösung
Connection refused :7687	Memgraph-Container down	<code>docker compose -f deploy/docker-compose.yml up -d</code>
Leerer Graph	Kein <code>cryme init</code>	<code>cryme init</code>
Lab zeigt alten Stand	Cache / alte Session	Seite neu laden, Query erneut ausführen
CLI findet Memgraph nicht	Port nicht erreichbar	<code>bash deploy/check_memgraph.sh</code>
HEAD ≠ erwarteter Schritt	Falsche Schrittnummer	<code>cryme show tree</code>
Memgraph ≠ Live-API	Deploy vergessen	<code>cryme deploy step=N</code>

11. CLI ↔ Memgraph Zuordnung

CLI-Befehl	Memgraph-Operation
<code>cryme init</code>	Graph leeren + YAML laden
<code>cryme show node</code>	<code>MATCH (n:CryptoAsset SecurityControl) ...</code>
<code>cryme show tree</code>	<code>MATCH (s:MigrationStep) ... + TRANSITION_TO</code>
<code>cryme show state step=N</code>	Replay via <code>reconstructStateAtStep()</code>
<code>cryme show graph step=N</code>	Replay + Kanten rendern
<code>cryme migrate ...</code>	Schreibt <code>MigrationStep</code> , aktualisiert Knoten
<code>cryme deploy step=N</code>	Liest Replay-Zustand, schreibt nicht in Memgraph

12. Wie die Beispiele zum Code passen

Demo-Beispiel	In Memgraph ansehen	Im Code geschrieben von
Schritt 0 <code>cryme init</code>	Component / CryptoAsset / SecurityControl / PQCVariant	<code>twin_loader.js</code> via <code>initDatabase()</code>
Schritt 1 Fail	<code>MigrationStep.status = failed</code> , <code>discovered_edge</code> , neue <code>IMPLICIT_DEPENDENCY</code>	<code>migrateNodes()</code> + <code>checkOracleValidation()</code>
Schritt 2 Success	beide KEX <code>status = migrated</code> , <code>HEAD = 2</code>	dieselben Funktionen; <code>updateHeadStep(2)</code>

Demo-Beispiel	In Memgraph ansehen	Im Code geschrieben von
Schritte 3–4	Cert + SecurityControl active_algorithm	derselbe Schreibpfad, andere Knoten-IDs
cryme show state step=N	rekonstruiert, kein gespeicherter Snapshot	reconstructStateAtStep() in oracle.js
cryme deploy	Graph unverändert	Ansible liest nur Replay-Ausgabe

13. Weiterführend

Dokument	Inhalt
docs/GRAPH_VERSIONING.md	Event Sourcing, HEAD, Replay-Algorithmus
docs/MIGRATION_STATES.md	Zustandsdiagramme pro Schritt
docs/migration_explanation.md	Oracle-SCC-Verhalten
docs/TECHNISCHE_DOKUMENTATION.md	Code-Architektur
docs/LIVE_DEMO_CHEAT_SHEET.md	5-Minuten-Demo zum Ausdrucken